

Software Engineering Group
Heidelberg University

Prof. Dr. Sebastian Baltes
Tobias Reichel

Advanced Software Engineering (IASE)
Assignment 2

FIFA World Cup 2026 Betting

Flaky Tests & BettingService Extension

Submitted by:

Rohit Sonejee Matrikelnummer: 4756184

Thanh Tran Matrikelnummer: 4209012

Group 18

Summer Semester 2026

May 16, 2026

GitHub Repository:

<https://github.com/Ravanoid3/Advanced-Software-Engineering>

Declaration of GenAI Use

We, **Rohit Sonejee** (ID 4756184) and **Thanh Tran** (ID 4209012), declare that we used generative AI tools in the development of this assignment. Specifically, we used **Kimi K2.5** and **Claude** (Anthropic). Below is a transparent breakdown of what was done independently versus with AI assistance.

Parts Completed Independently

- Identification and analysis of the five flaky tests through manual inspection and repeated runs of `./gradlew repeatTests`.
- Design and implementation of all five fixes in the source code.
- Implementation of `evaluateBonus`, `removeBet`, and `changeBet` in `BettingService.kt`.
- Writing of the eight JUnit test stubs in `BettingServiceTest.kt`.

AI-Assisted Parts

- **Kimi K2.5 & Claude** – used to help phrase and structure explanations in the flaky test report, and to cross-check the completeness of test coverage.

Exercise 2.1 – Flaky Tests

(a) Flaky Test Report

Flaky Test 1

Test name:

```
de.seuhd.worldcup.FileBettingServiceTest#test_file_betting_with_threads
```

Root cause:

The test concurrently writes bets to the same file from two different threads, which may lead to race conditions during read and write operations. Depending on thread scheduling, updates could overwrite each other, resulting in missing bets.

Fix:

We synchronized all file access in `FileBettingService` using a shared lock object. Both `placeBet()` and `getBets()` now execute atomically, preventing race conditions and ensuring that all bets are written and read consistently regardless of thread timing.

Flaky Test 2

Test name:

```
de.seuhd.worldcup.FileBettingServiceTest#fresh_service_has_no_bets
```

Root cause:

Multiple tests shared the same persistent file (`SHARED_BET_FILE`). Since the test class uses `@TestMethodOrder(MethodOrderer.Random::class)`, the execution order changes between runs. If another test wrote bets into the shared file before this test executed, the file was no longer empty.

Fix:

We added a cleanup function that deletes the shared file after each test execution using `@AfterEach`. This ensures that every test starts with a clean and isolated file state, making the tests independent of execution order.

Flaky Test 3

Test name:

```
de.seuhd.worldcup.WorldCupTest#standings_are_stable_when_multiple_teams_tie_on_all_criteria
```

Root cause:

When teams had identical points, goal difference, and goals scored, the standings calculation did not define a final deterministic tie-breaker. The remaining order depended on the iteration order of `IdentityHashMap` and, as a result, teams occasionally appeared in a different order across runs.

Fix:

We added a final deterministic sorting criterion (`team.name`) to the standings comparator, which eliminates the non-deterministic iteration behavior.

Flaky Test 4**Test name:**`de.seuhd.worldcup.WorldCupTest#load_json_from_network`**Root cause:**

The test depends on real network calls and a randomly shuffled URL list. This makes the outcome non-deterministic due to network variability and unpredictable URL selection order.

Fix:

We replaced the real network call with a test double that loads local JSON data, in order to remove non-determinism by simulating a real network call. Since we used local data, it does not affect the test result. For the sake of removing order dependencies we also removed the shuffling of the URLs, especially since one of the provided URLs does not work. We also increased the timeout value; otherwise the test kept failing.

Flaky Test 5**Test name:**`de.seuhd.worldcup.WorldCupTest#evaluate_returns_zero_when_no_bets_are_placed`**Root cause:**

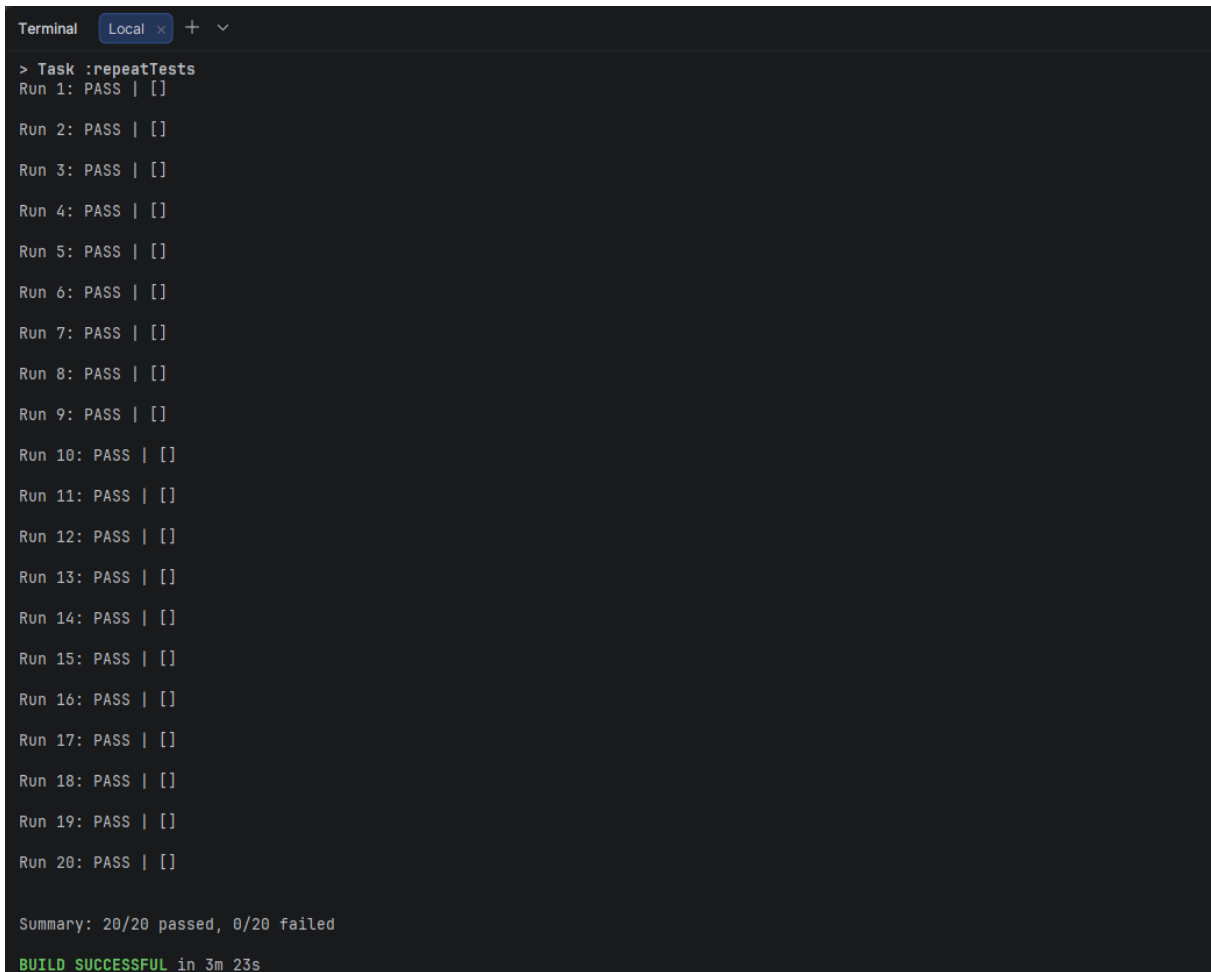
`BettingService` cached evaluation results in `cachedResult`, but the cache was not cleared when stored bets were removed. Since the cached value persisted between tests, later tests could reuse outdated evaluation results from previous executions, depending on test order.

Fix:

We reset `cachedResult` inside the `clear()` method. This ensures that each test starts with a clean state and that evaluations are always recomputed from the current set of bets instead of reusing outdated cached results.

(b) Fix – ./gradlew repeatTests Output (0 Failures)

The screenshot below shows the terminal output after all fixes were applied, confirming zero failures across all 20 repetitions.



```
Terminal Local x + v
> Task :repeatTests
Run 1: PASS | []

Run 2: PASS | []

Run 3: PASS | []

Run 4: PASS | []

Run 5: PASS | []

Run 6: PASS | []

Run 7: PASS | []

Run 8: PASS | []

Run 9: PASS | []

Run 10: PASS | []

Run 11: PASS | []

Run 12: PASS | []

Run 13: PASS | []

Run 14: PASS | []

Run 15: PASS | []

Run 16: PASS | []

Run 17: PASS | []

Run 18: PASS | []

Run 19: PASS | []

Run 20: PASS | []

Summary: 20/20 passed, 0/20 failed
BUILD SUCCESSFUL in 3m 23s
```

Figure 1: `./gradlew repeatTests` – 0 failures across 20 runs

Exercise 2.2 – BettingService Extension

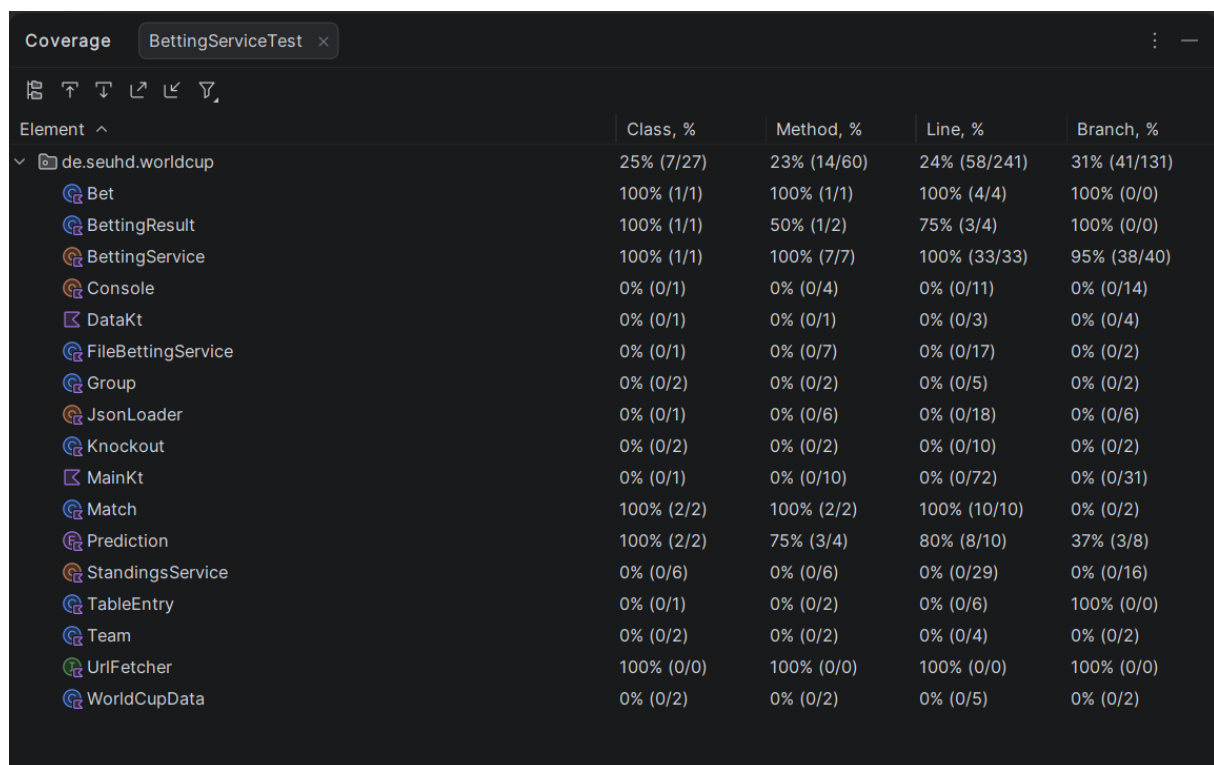
(a) Implementation

We implemented the three method stubs in `BettingService.kt`:

- `evaluateBonus(matches)`: Iterates over all stored bets and computes a bonus score per bet. A bet scores 3 points when both `predictedHomeScore` and `predictedAwayScore` exactly match the actual final scores. It scores 1 point when only the predicted outcome (home win, away win, or draw) matches. Otherwise it scores 0 points. The method sums and returns the total.
- `removeBet(matchId)`: Looks up the internal collection for the given `matchId` and removes the corresponding bet. If no such bet exists the method does nothing (no exception is thrown).
- `changeBet(bet)`: Replaces an existing bet that matches `bet.matchId` with the supplied `bet` object. Throws `IllegalArgumentException` if no bet for that match exists.

(b) Tests & Coverage

We completed all eight `@Test` stubs in `BettingServiceTest.kt`. The IntelliJ coverage report shown below confirms that the tests achieve above the required 80 % threshold for method, line, and branch coverage (95 % – 38 / 40 methods covered).



The image shows the IntelliJ Coverage tool window for the test class BettingServiceTest. The table displays coverage metrics for various elements in the project de.seuhd.worldcup. The BettingService class is highlighted with a 95% method coverage rate.

Element	Class, %	Method, %	Line, %	Branch, %
de.seuhd.worldcup	25% (7/27)	23% (14/60)	24% (58/241)	31% (41/131)
Bet	100% (1/1)	100% (1/1)	100% (4/4)	100% (0/0)
BettingResult	100% (1/1)	50% (1/2)	75% (3/4)	100% (0/0)
BettingService	100% (1/1)	100% (7/7)	100% (33/33)	95% (38/40)
Console	0% (0/1)	0% (0/4)	0% (0/11)	0% (0/14)
DataKt	0% (0/1)	0% (0/1)	0% (0/3)	0% (0/4)
FileBettingService	0% (0/1)	0% (0/7)	0% (0/17)	0% (0/2)
Group	0% (0/2)	0% (0/2)	0% (0/5)	0% (0/2)
JsonLoader	0% (0/1)	0% (0/6)	0% (0/18)	0% (0/6)
Knockout	0% (0/2)	0% (0/2)	0% (0/10)	0% (0/2)
MainKt	0% (0/1)	0% (0/10)	0% (0/72)	0% (0/31)
Match	100% (2/2)	100% (2/2)	100% (10/10)	0% (0/2)
Prediction	100% (2/2)	75% (3/4)	80% (8/10)	37% (3/8)
StandingsService	0% (0/6)	0% (0/6)	0% (0/29)	0% (0/16)
TableEntry	0% (0/1)	0% (0/2)	0% (0/6)	100% (0/0)
Team	0% (0/2)	0% (0/2)	0% (0/4)	0% (0/2)
UrlFetcher	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
WorldCupData	0% (0/2)	0% (0/2)	0% (0/5)	0% (0/2)

Figure 2: IntelliJ coverage report – 95 % method coverage for BettingService